

MAP08

Robustesse des grands modèles d'apprentissage

El Kabid Abdessamad,
Benmansour Mohammed,
Mok Yong,
Louis Paul-Philip

24 avril 2024

Tuteur : El Mhamdi El Mahdi – Coordinateur : Pichard Teddy.
Établissement d'accueil : CMAP École polytechnique.

Table des matières

1	Introduction	3
2	État de l'art	5
3	Notations et définitions	7
3.1	Définitions générales en machine learning	7
3.1.1	Vocabulaire des attaques	9
4	Expériences sur l'influence de la dimension	10
4.1	Régression linéaire et polynomiale	10
4.2	Support Vector Machines - SVM	13
4.3	Neural Networks	15
4.4	Premières conclusions	19
5	Étude mathématique des attaques	20
5.1	Méthode d'attaque colinéaire :	21
5.2	Méthode d'attaque orthogonale :	22
6	Expériences sur les attaques	23
6.1	Description de l'expérience	23
6.2	Exécution et résultats des expériences	24
6.3	Synthèse des observations	25
7	Faiblesse des modèles sans défense	26
8	Conclusion	29

1 Introduction

Ce PSC vise à étudier la robustesse de divers modèles d'apprentissage face à des attaques, et le compromis entre la performance permise par une grande dimension du modèle, et la perte en fiabilité causée par une telle dimension.[1, 7]

Les études existantes [1, 2, 5, 7] convergent vers l'idée d'une relation directe entre la taille des modèles et leur vulnérabilité potentielle en termes de robustesse et de préservation de la confidentialité des données. Prolonger cette analyse nécessite l'examen des développements récents dans les domaines de la sécurité des modèles d'IA, de la certification des données, des méthodes d'atténuation des risques d'empoisonnement des données, et des techniques de préservation de la confidentialité. L'avenir de la fiabilité des modèles d'IA dépendra de la capacité à résoudre ces défis complexes tout en tirant parti du potentiel des modèles de grande envergure dans le domaine de l'intelligence artificielle.

Nous sommes partis d'une conjecture, exposée ci-dessous.

On note, en fonction du nombre de paramètres d (neurones, biais, paramètres de régression....) du modèle, $t_0(d)$ le taux de prédictions correctes du modèle entraîné sans attaque des données, $t_1(d)$ ce taux sous attaque des données avec une protection (défense, voir [1]), et $t_2(d)$ ce taux avec attaque sur les données et sans défense.

1. $t_0(d)$ augmenterait jusqu'à un certain seuil, au-delà duquel il diminuerait à mesure que le nombre de paramètres augmente (risque de surajustement (*over-fitting*)). Il suivrait par ailleurs asymptotiquement une tendance décroissante en $\frac{1}{\sqrt{d}}$.
2. Sur des petites dimensions, $t_1(d)$ suivrait une tendance similaire à $t_0(d)$, mais resterait inférieur (le modèle sous attaque n'est pas supposé mieux performer sur de vraies données que le modèle sans attaque). L'écart entre les deux dépendrait de l'efficacité des attaques et de la robustesse du modèle face à ces attaques. Par la suite, la courbe atteindrait un maximum puis décroîtrait, et suivrait asymptotiquement une décroissance en $\frac{1}{\sqrt{d}}$.
3. Le maximum de $t_1(d)$ serait atteint avant celui de $t_0(d)$.
4. $t_2(d)$ s'aplatit et peut même être contrôlé en fonction du type d'attaque.

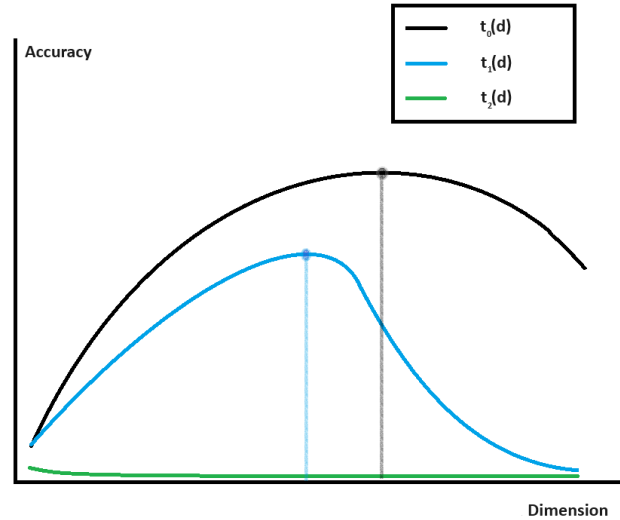


FIGURE 1 – Conjecture sur les performances des modèles (en *test accuracy*) en fonction du degré du polynôme.

Les contribution de PSC sont les suivantes :

- Nous avons étudié, pour différents modèles (régression linéaire, SVM, réseau de neurones) l'influence de la dimension de l'espace d'entraînement sur la précision du modèle.
- Nous avons établi mathématiquement un parallèle entre le *label flipping* et l'attaque de gradient, ce qui nous a permis de développer une stratégie d'attaque sur le jeu de données qui perturbe efficacement la descente de gradient.

Ce rapport se structure comme suit : la section 2 présente l'état de l'art en la matière, ainsi qu'un recueil en 3 de définitions des notions-clés de ce rapport. Nous présentons ensuite, en section 4, les expériences effectuées avec différents modèles d'entraînement : régression polynomiale, SVM à noyau polynomial, et réseaux de neurones. Afin d'acquérir une meilleure compréhension des différentes attaques, nous nous sommes focalisés sur des expériences avec attaque uniquement. Les résultats obtenus contredisant la conjecture initiale, plutôt que de nous intéresser à des stratégies de défense, nous avons préféré prendre de la hauteur mathématique sur les attaques dans la section 5. Cela nous a permis d'établir une stratégie d'attaque entre attaque de gradient et *label flipping* (voir ci-dessous) que nous avons expérimenté en 6.

En outre, nos résultats pourront servir de meilleur point de départ à un future PSC ou un EA recherche sur le sujet. Enfin, la section 7 explore des pistes de défense que nous n'avons pas eu le temps d'approfondir.

2 État de l'art

La place des algorithmes de *machine learning* dans le monde a connu une progression spectaculaire au cours des dernières décennies. Leur essor a permis l'émergence d'applications allant de la reconnaissance vocale à la conduite autonome, en passant par la prédiction de maladies. À mesure que les systèmes d'intelligence artificielle sont intégrés dans des domaines critiques de la société, tels que la santé, la finance, la sécurité publique et la prise de décision, ces avancées ne sont pas sans susciter des interrogations quant à la fiabilité de ces algorithmes. Les intelligences artificielles conçues par *machine learning* étant entraînées sur des jeux de données, des erreurs dans les données fournies peuvent dégrader les conditions d'apprentissage, et nuire en définitive à la qualité de ces algorithmes. Les jeux de données apparaissent dès lors comme une cible de choix pour les cyberattaques de demain. C'est là qu'intervient la notion de robustesse en *machine learning*.

Cette dernière décrit la capacité d'un modèle d'intelligence artificielle à maintenir des performances élevées et fiables, même dans des conditions d'entraînement dégradées pouvant faire l'objet d'attaques malveillantes. Cela inclut notamment des données bruitées, des données biaisées, des *outliers* (exemples aberrants, ie très différents du reste des données), ou encore des injections de gradients erronés lors du calcul d'une descente de gradient.

Il importe de comprendre les répercussions de ces types de dégradation sur les performances des différents modèles de *machine learning*, afin de mieux s'en prémunir, et de développer des stratégies pertinentes de défense, qui puissent renforcer la robustesse des modèles.

La recherche en matière de fiabilité des modèles d'apprentissage utilisés en intelligence artificielle (IA) met en évidence l'impossibilité croissante de concilier la grande taille des modèles avec leur fiabilité. Cette complexité se décline en plusieurs sous-aspects cruciaux qui soulèvent des questions fondamentales dans le domaine de l'IA :

- **Robustesse face aux attaques** : Un défi majeur réside dans la garantie de la robustesse des modèles d'apprentissage face aux données compromises et aux acteurs malveillants cherchant à les corrompre. Les attaques orchestrées par des acteurs Byzantins, qu'il s'agisse de la falsification de données ou de comportements malveillants, menacent la fiabilité des modèles. Le développement de mécanismes de défense tels que l'agrégation Krum est essentiel pour atténuer ces menaces et assurer la stabilité des modèles [1, 7].
- **Entraînement des Large AI Models (LAIMs) avec des données non certifiées** : Un autre défi significatif découle de la nécessité d'entraîner des modèles d'IA de grande envergure avec une quantité massive de données. Dans la réalité, il est souvent impossible de certifier ou de vérifier toutes les données utilisées. Par exemple, pour l'entraînement d'un modèle de langage, il est peu réaliste de s'appuyer uniquement sur des textes certifiés compte tenu du volume colossal de données nécessaires [2].
- **Impact de la dimension des modèles sur la précision** : Il est crucial de noter que l'augmentation de la dimension des modèles entraîne généralement

une amélioration de leur précision. Toutefois, toute réduction de dimension a un impact significatif sur la précision. Les modèles performants tendent à avoir des dimensions de l'ordre de 10^{11} à 10^{14} , se rapprochant des 10^{15} synapses présentes dans le cerveau humain. Le défi réside dans le fait qu'à grande dimension, les risques d'empoisonnement des données et de propagation d'informations erronées augmentent, compliquant la garantie de fiabilité des modèles [2].

- **Préservation de la confidentialité des données privées :** Le défi de la préservation de la confidentialité des données sensibles utilisées dans le processus d'entraînement des modèles d'IA est essentiel à comprendre en profondeur. À mesure que la taille des modèles augmente, il devient de plus en plus complexe de garantir que les données privées demeurent confidentielles. Cette complexité découle du fait que, avec des modèles de grande envergure, il existe un risque accru d'accès non autorisé ou de divulgation des données originales utilisées pour l'entraînement du modèle. Cette préoccupation est cruciale car elle touche à la vie privée des individus et à la protection de leurs informations sensibles contre tout accès indésirable ou utilisation abusive [5].

3 Notations et définitions

3.1 Définitions générales en machine learning

Modèle

Un modèle de *machine learning* est une représentation mathématique d'un processus complexe, utilisée pour faire des prédictions ou des classifications sur de nouvelles données. Entraîné sur un jeu de données, ou *dataset*, le modèle ajuste ses paramètres pour minimiser les erreurs entre ses prédictions et les valeurs réelles. Une fois entraîné, il peut être utilisé pour prédire des résultats sur de nouvelles données, son efficacité dépendant de la qualité des données d'entraînement et de l'algorithme d'entraînement.

Dataset

Un *dataset*, ou jeu de données, est une collection structurée de données utilisée pour entraîner, valider et tester des modèles d'apprentissage automatique. Il contient généralement des exemples individuels, où chaque exemple est composé de caractéristiques ou attributs pertinents et de la valeur cible à prédire ou à classer. Ces données peuvent provenir de diverses sources telles que des expériences en laboratoire, des observations sur le terrain ou des enregistrements numériques. Un *dataset* est fondamental pour le processus d'apprentissage automatique car il fournit au modèle les informations nécessaires pour apprendre à partir des données et généraliser des modèles prédictifs ou classificateurs à de nouvelles situations.

Un exemple de *dataset* est le *dataset* MNIST, qui contient 70000 images de 28x28 pixels, représentant des chiffres manuscrits, de 0 à 9, avec le nombre correspondant.



FIGURE 2 – Exemple des images contenues dans le *dataset* MNIST

Loss function

Une fonction de perte, également appelée *loss function* en anglais, est un outil crucial en *machine learning*. Elle évalue l'écart entre les prédictions d'un modèle et les valeurs réelles qu'il devrait produire. Imaginez un modèle tentant de prédire le prix des maisons en fonction de caractéristiques telles que la taille et l'emplacement. La fonction de perte mesurera à quel point les prédictions du modèle à partir de ces caractéristiques s'écartent des prix de réelles maisons. Un autre exemple serait dans la classification d'images : si un modèle essaie de déterminer si une image montre

un chat ou un chien, la fonction de perte évaluera à quel point les prédictions du modèle correspondent aux étiquettes réelles des images.

En ajustant ses paramètres lors de l'entraînement, le modèle cherche à minimiser cette fonction de perte. Par exemple, dans le cas de la régression, une fonction de perte couramment utilisée est l'erreur quadratique moyenne (Mean Squared Error), qui calcule la moyenne des carrés des écarts entre les prédictions et les valeurs réelles. Pour la classification, l'entropie croisée est souvent utilisée, mesurant la différence entre les probabilités prédites par le modèle et les probabilités réelles des classes. En minimisant cette fonction de perte, le modèle apprend à améliorer ses prédictions et à mieux généraliser à de nouvelles données.

Entraînement

L'entraînement en *machine learning* désigne le processus par lequel un modèle est ajusté aux données d'entraînement afin d'apprendre à faire des prédictions ou des classifications. Pendant cette phase, le modèle ajuste ses paramètres internes pour minimiser une fonction de perte, qui mesure l'écart entre ses prédictions et les valeurs réelles tirées d'un *dataset* d'entraînement (le *training set*). En d'autres termes, le modèle apprend à partir des données en cherchant à réduire les erreurs qu'il commet lors de ses prédictions.

L'entraînement implique souvent plusieurs itérations, aussi appelés **epochs**, sur le *dataset* d'entraînement, où le modèle est exposé à différents exemples et ajuste ses paramètres en conséquence. Ce processus peut être itératif et nécessite souvent l'utilisation d'algorithmes d'optimisation pour trouver les valeurs optimales des paramètres du modèle. Une fois l'entraînement terminé, le modèle est évalué sur un ensemble de données de validation pour évaluer ses performances et ajuster les hyperparamètres si nécessaire. En fin de compte, l'objectif de l'entraînement est de créer un modèle capable de généraliser à de nouvelles données et de produire des prédictions précises.

Accuracy

La précision (*accuracy*) en *machine learning* mesure la proportion de prédictions correctes réalisées par un modèle sur un ensemble de données donné. Elle est souvent utilisée pour évaluer les performances d'un modèle de classification. Une *accuracy* de 1 signifie que toutes les prédictions sont correctes, tandis qu'une *accuracy* de 0 indique que toutes les prédictions sont incorrectes. La précision est un indicateur important pour évaluer la qualité générale d'un modèle, mais elle peut être biaisée dans des cas où les classes sont déséquilibrées.

On peut aussi distinguer *training accuracy* et *test accuracy*, qui dépend de la portion du *dataset* (*train* ou *test*) utilisée pour l'évaluer.

Note : Ces concepts seront explorés plus en profondeur au fil des expériences et des démonstrations présentées dans ce rapport.

3.1.1 Vocabulaire des attaques

Dans le domaine de *machine learning*, la sécurité des modèles est une préoccupation majeure en raison de divers types d'attaques qui peuvent compromettre leur robustesse. Deux de ces attaques sont particulièrement pertinentes : le *label flipping* et l'attaque de gradient.

Label flipping

Le *label flipping* implique une manipulation délibérée des étiquettes (labels) des données d'entraînement, induisant ainsi le modèle à apprendre des prédictions incorrectes. Ce type d'attaque peut être utilisé pour induire des biais dans le modèle, entraînant des décisions erronées.

Attaque de gradient

L'attaque de gradient vise à altérer les gradients calculés pendant l'entraînement en injectant des données malveillantes ou en modifiant les gradients existants. Ces modifications peuvent perturber le processus d'optimisation du modèle, conduisant à des résultats finaux moins fiables voire à des modèles complètement compromis.

Poison rate

Le *poison rate*, ou taux d'empoisonnement est la fraction des données qui est empoisonnée. Exemple : un *poison rate* de 10% sur un *dataset* de 10000 points signifie que 1000 points ont été empoisonnés.

4 Expériences sur l'influence de la dimension

Durant ces premières expériences, nous essayons de vérifier la conjecture décrite plus haut. Pour cela, nous avons expérimenté avec différents modèles en variant la dimension et le *poison rate*.

4.1 Régression linéaire et polynomiale

Description de l'expérience

Dans cette expérience, nous allons utiliser les modèles de régression linéaire et polynomiale sur le *dataset* MNIST décrit ci-dessus.

Il faut noter que $d \geq 1$ représente dans cette expérience le degré du polynôme utilisé.

Langage utilisé : Python

Librairie utilisée : numpy, pytorch, datasets

Base de données : pour la sauvegarde et le traitement des données, MySQL enregistrant pour l'*accuracy* avec la *loss function* et l'itération de sauvegarde.
→ L'enregistrement prend en compte le modèle utilisé, la dimension, le type d'empoisonnement (ex : *Label Flipping*), le *poison rate*, et le numéro de l'essai.

Algorithme : Voici l'algorithme utilisé pour l'expérience :

1. **Adapter le *dataset* :** La première partie de l'algorithme est de transformer l'ensemble des images en vecteurs normalisés.

On commence par aplatir les images, c'est-à-dire les transformer d'une matrice de luminosité en vecteurs, en concaténant les lignes de la matrices.

Ensuite on normalise en divisant par 256 (Les valeurs initiales étaient de 0 à 255).

Un problème se pose toutefois : les pixels des images du *dataset* MNIST sont majoritairement noirs ou blancs, donc sont représentés par des 0 et des 1 qui restent inchangés par exponentiation. La régression polynomiale, appliquée telle quelle, serait donc inefficace. Pour y remédier, nous avons effectué une série d'additions (cf p13). On note x_k la k -ième valeur du vecteur image x normalisé de dimension $28 \times 28 = 784 = n$, et x^{new} le nouveau vecteur de même dimension :

$$\forall k \in \llbracket 1, n \rrbracket : x_k^{\text{new}} = \sum_{i=\max(1, k-d)}^k x_i$$

Ainsi, par exemple pour $d = 3$, on aura $x_1^{\text{new}} = x_1$, $x_2^{\text{new}} = x_1 + x_2$,

$x_3^{\text{new}} = x_1 + x_2 + x_3$, $x_4^{\text{new}} = x_2 + x_3 + x_4$, ...

A la fin, pour s'assurer qu'on garde la normalisation, on divise le tout par d .

La dernière étape est de transformer ce vecteur en vecteur polynomial, c'est à dire de faire la **concaténation** suivante :

$$x \leftarrow [x, x^2, x^3, \dots, x^d]$$

Note : En réalité, pour le degré $d = 2$, c'est le terme $\frac{1}{1+10.x}$ qui a été employé au lieu de x^2 , ainsi entraînant un décalage. L'idée n'était pas de seulement utiliser des polynômes, mais aussi d'autres fonctions non polynomiales, comme $\frac{1}{x}$. Pour éviter d'avoir un dénominateur nul, et pour que les nouvelles valeurs s'étendent sur $[0.1, 1]$, on a opté pour la formule ci-dessus.

$$x \leftarrow \left[x, \frac{1}{1 + 10.x}, x^2, x^3, \dots, x^{d-1} \right]$$

2. **Label Flipping :** Soit $0 < r < 1$ le taux d'empoisonnement. Si $r = 0$, cette étape est ignorée.

On prend les $\lfloor r.N \rfloor$ premières données, N étant le nombre total de données. Ensuite on change les labels, possiblement de manière aléatoire, mais nous avons préféré le faire de la manière suivante, pour avoir (intuitivement) plus d'effet et être contrôlable :

$$\forall i \in \llbracket 1, \lfloor r.N \rfloor \rrbracket : y_i \leftarrow (y_i + 1) \mod 10$$

3. **Entraînement :** L'entraînement consiste à exécuter un algorithme d'optimisation sur plusieurs itérations, l'algorithme utilisé ici est le *Adam Optimization Algorithm* avec un *learning rate* de 0.001. Le choix du nombre d'itérations dépend de l'éventuelle convergence de la précision du modèle sur l'échantillon test. Cela prend généralement quelques dizaine de milliers d'itérations, rarement des centaines de milliers.
4. **Évaluation des performances :** Pour évaluer les performances, après chaque 200 itérations, on sauvegarde dans la base de données la *loss function* et la *test accuracy*. Pour évaluer l'*accuracy*, la vérification d'un label se fait en prenant le chiffre avec la plus grande probabilité. On note y_i le vrai label, et $\hat{y}$ le vecteur de dimension 10 contenant les *probabilité* prédite de chaque chiffre. On vérifie donc si

$$y_i = \operatorname{argmax}_{i \in \llbracket 1, 10 \rrbracket} \hat{y}_i$$

Exécution du code et traçage de la courbe

Nous avons exécuté l'algorithme précédent pour $d \in \llbracket 1, 60 \rrbracket$, et pour le *poison rate* $r \in \{0, 0.1, 0.2, 0.3, 0.4\}$. Nous avons pris pour chaque test la valeur maximale de la *test accuracy* parmi celle enregistrée chaque 200 itérations.

Note : Ici, la dimension représente le nombre de paramètre du modèle. En la notant D , et en ajoutant 10 paramètres pour inclure le biais, on a :

$$D = 28 * 28 * 10 * d + 10$$

Car la sortie du modèle est un vecteur de probabilités d'appartenance à chacune des 10 classes .

Voici le résultat :

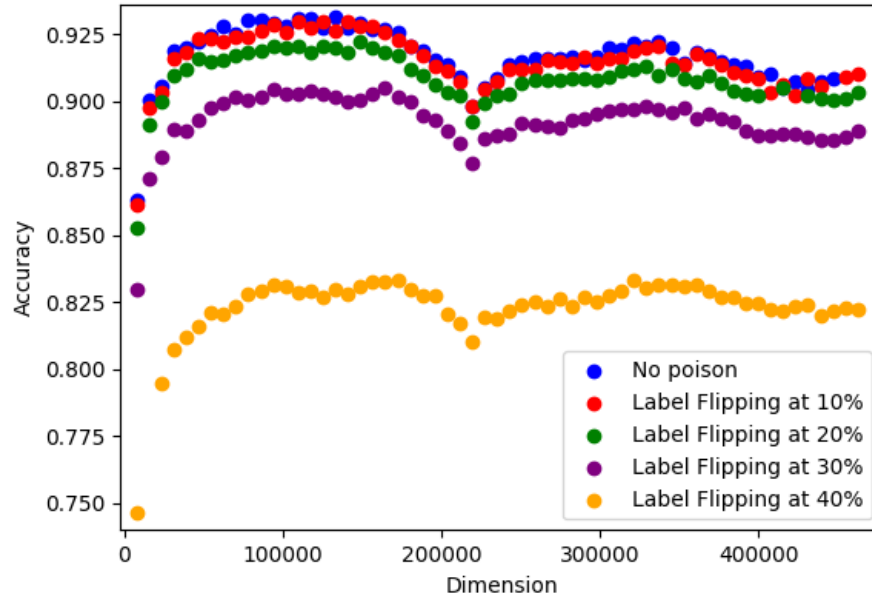


FIGURE 3 – La *test accuracy* des différents modèles de régression polynomiale sur MNIST pour différentes valeurs du taux de poison en fonction de la dimension.

Analyse des résultats

On remarque tout d'abord qu'il y a une translation presque fixe entre les différentes courbes pour différents *poison rates*. Cet écart croît de plus en plus vite à mesure que le taux d'empoisonnement augmente.

Ensuite on remarque que les courbes ont tous un pic aux alentours de la dimension 219520, qui est celle du polynôme de degré 28. Or cela nous rappelle la dimension des images qui est 28x28.

Nous avons réfléchi à de potentielles causes de ce pic. Cela peut être dû aux séries d'additions effectuées pour améliorer la régression polynomiale. En effet, ces additions avaient pour but de diversifier les valeurs en entrée en associant les pixels voisins de la même ligne (avec un retour à la ligne à la fin). Ainsi pour $d = 28$, ces sommes s'effectuaient sur une ligne complète, ce qui n'était plus forcément pertinent, et susceptible de dégrader la qualité du modèle. Et pour $d > 28$, on incluait le pixel en-dessous du pixel précédent, qui pouvait être utile dans ce traitement d'image.

Voici une représentation graphique pour mieux le concevoir. Pour $d = 1$, on a le seul pixel bleu. Ensuite pour $1 < d < 28$, on calcule la somme des d pixels : le bleu et les $d - 1$ pixels suivants dans le vert. Pour $d = 28$ on a toute la première ligne. Pour $d = 29$, on inclut le pixel sous le bleu, qui est donc voisin du bleu.



FIGURE 4 – Représentation graphique de la somme des pixels pour différents d .

Cela suggère la recherche d'une relation fonctionnelle entre l'*accuracy* et le taux d'empoisonnement.

Toutefois, les résultats obtenus sur ce sujet ne sont pas concluants à ce stade.

4.2 Support Vector Machines - SVM

Description de l'expérience

Après le cas de la régression, nous avons choisi de nous intéresser aux modèles de SVM avec noyaux polynomiaux, toujours avec le but d'étudier l'influence de la dimension du modèle et du taux de *label flipping*.

Langage utilisé : Python

Librairies utilisées : Numpy, scikit-learn

Datasets : MNIST et *digits* de scikit-learn (une version miniature de MNIST, contenant 1797 images en 8x8 pixels).

Nous avons directement utilisé la méthode `sklearn.svm.SVC` avec comme paramètre de régularisation $C = 1$ et avec un noyau polynomial, dont nous avons varié le degré.

Première expérience avec SVM

Ci-dessous 5, le graphique obtenu pour le *dataset digits*, en faisant varier le degré du noyau polynomial du modèle de 1 à 80 avec différents taux de *label flipping* allant de 0% (sans attaque) à 50%.

Le *label flipping* a été effectué en attribuant de manière aléatoire un *label* entre 0 et 9 à une certaine proportion du *training set*.

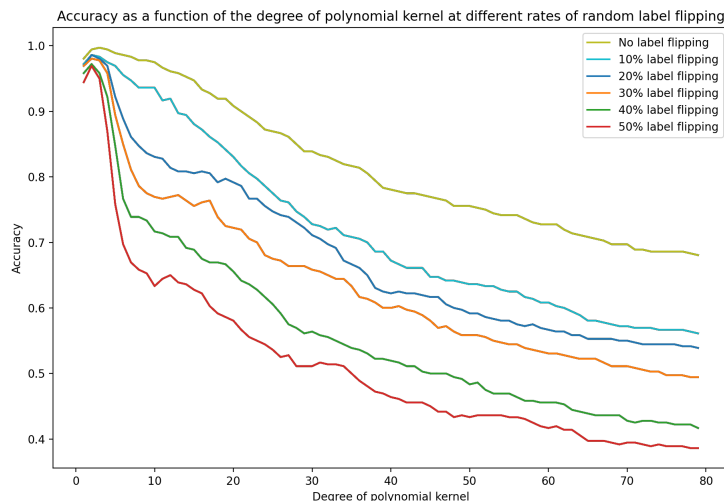


FIGURE 5 – *Accuracy* du modèle en fonction de la dimension du noyau polynomial à différents taux de *label flipping*

Analyse du premier résultat

Nous remarquons d’abord qu’il y a une décroissance similaire à celle observée en régression lorsque nous augmentons le taux de *label-flipping*; cependant, cette baisse est presque constante d’un taux de *label-flipping* à l’autre.

Deuxièmement, nous constatons que pour chaque taux de *label-flipping*, l’*accuracy* augmente initialement très rapidement lorsque le degré du noyau polynomial s’accroît. Elle atteint ensuite un sommet autour des degrés 5 ou 6. À partir de ce point, elle décroît à un taux qui est approximativement inversement proportionnel à la racine carrée du degré $1/\sqrt{d}$. En effet, l’utilisation de degrés de noyau polynomial plus élevés peut potentiellement conduire à un sur-apprentissage (*overfitting*), ce qui réduit la performance du modèle.

Deuxième expérience avec SVM

Nous avons également conduit notre expérimentation sur le SVM avec le *dataset* MNIST sans procéder à du *label flipping*. Ici, notre objectif est d’évaluer la performance du SVM avec d’autres *datasets* afin de déterminer les points communs et les différences entre ces ensembles de données. En augmentant le degré du noyau polynomial de 1 à 39 sur le *dataset* MNIST, nous avons obtenu les résultats illustrés dans la figure ci-dessous :

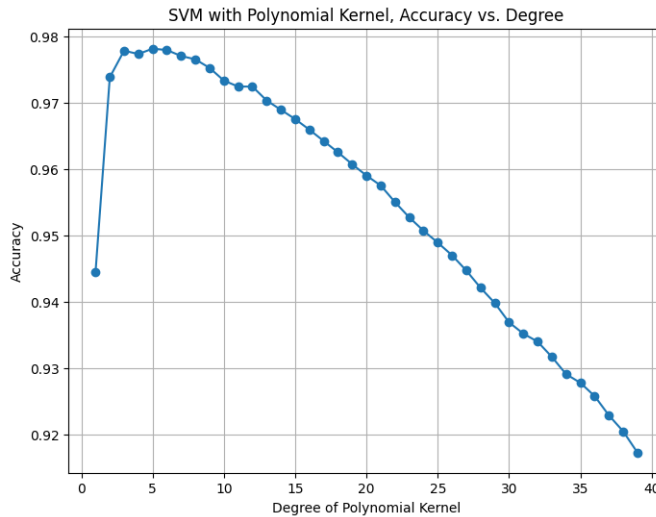


FIGURE 6 – *Accuracy* du modèle en fonction de la dimension du noyau polynomial

Analyse du deuxième résultat

Dans la figure 6, nous observons un pic d'*accuracy* qui est rapidement atteint pour des noyaux polynomiaux d'environ 3 à 6 degrés, avec une *accuracy* avoisinant les 98%. Par la suite, l'*accuracy* décroît de manière presque linéaire, descendant en dessous de 92% pour un degré de 39. Avec le *dataset* MNIST, nous remarquons une baisse linéaire de l'*accuracy*. Cela n'est pas concluant vis-à-vis de la conjecture initiale (nous attendions une décroissance proportionnelle à $1/\sqrt{d}$).

Toutefois, cette évolution évoque celle observée à de faibles degrés pour le *dataset digits* en figure 5, ce qui suggère que nous n'avons peut-être simplement pas atteint de degrés suffisamment élevés pour observer une telle évolution.

4.3 Neural Networks

Description de l'expérience :

Un réseau neuronal est un modèle d'apprentissage profond composé de différentes couches de neurones artificiels, chacun correspondant à un modèle simple (régression linéaire, logistique...). Il est facile à mettre en œuvre et on peut facilement calculer les dimensions du modèle. Dans le cas d'un réseau neuronal, l'augmentation de la dimension consiste simplement à augmenter le nombre de neurones dans chaque couche ou à augmenter la couche du modèle. Ici, nous nous intéressons à l'augmentation du nombre de neurones dans chaque couche cachée et à l'influence de la dimension du modèle et du taux de *label flipping* sur la précision.

Langage utilisé : Python

Librairies utilisées : Numpy, scikit-learn, pytorch

Datasets : MNIST et *digits* de scikit-learn (une version miniature de MNIST, contenant 1797 images en 8x8 pixels).

L'architecture de notre modèle :

la couche d'entrée : $64 = 8 \times 8$ neurones.

les couches cachées : 3 couches cachées avec des mêmes nombres de neurones.

La fonction d'activation : la fonction Unité Linéaire Rectifiée (ReLU) où $ReLU(x) = \max(0, x)$.

L'algorithme de notre expérience :

1. **Préparation du jeu de données :** Nous commençons par importer les données depuis la bibliothèque `scikit-learn`. Nous normalisons ensuite les données pour qu'elles soient comprises entre 0 et 1, nous les mélangeons en utilisant `random shuffle` et nous les partitionnons en un ensemble d'entraînement (80% des données) et en un ensemble de test (20% des données).
2. **Label flipping :** Soit r le taux d'empoisonnement de *label-flipping*. Si $r = 0$, il n'y a pas d'empoisonnement, sinon on prend $n = \lfloor r.N \rfloor$ le nombre de données empoisonnées dans le *training set*. Pour cela, on effectue une permutation circulaire des étiquettes entre les n derniers points.
3. **Entraînement :** Dans notre modèle, nous utilisons la fonction de perte d'entropie croisée couplée à l'algorithme d'optimisation de descente de gradient stochastique (SGD). Le modèle est entraîné sur 20 itérations avec un taux d'apprentissage de 0.01.
4. **Évaluation :** Pour évaluer notre modèle, nous testons notre modèle avec l'ensemble de test et comptons le nombre total de points de données que notre modèle prédit correctement. Ensuite, nous calculons la précision du modèle en divisant ce nombre par le nombre total de l'ensemble de test.
5. **Augmenter le nombre de neurones :** Nous augmentons le nombre de neurones dans les couches cachées et nous réentraînons et testons notre modèle.

Exécution du code et traçage de la courbe :

Dans notre expérience, nous utilisons le taux de *label-flipping* r de 0.0 à 1.0 et nous faisons varier le nombre de neurones dans chaque couche cachée, n_i de 64 à 72559 ($n_{i+1} = \lfloor 1.5n_i \rfloor$).

Remarque : la dimension représente le nombre total de paramètres du réseau de neurones. Soit n_r le nombre total de neurones dans chaque couche du réseau de neurones, donc on obtient le nombre total de paramètre D :

$$D = \sum n_{r+1}(n_r + 1) \quad \text{où } r \in \llbracket 0, 4 \rrbracket$$

Dans notre modèle $n_0 = 64$, $n_1 = n_2 = n_3$ et $n_4 = 10$.

Résultat :

Il y a deux images, la première 7 trace la courbe pour chaque taux de *label flipping* dans la même image et la seconde 8 trace chaque courbe séparément pour nous aider à analyser correctement nos résultats.

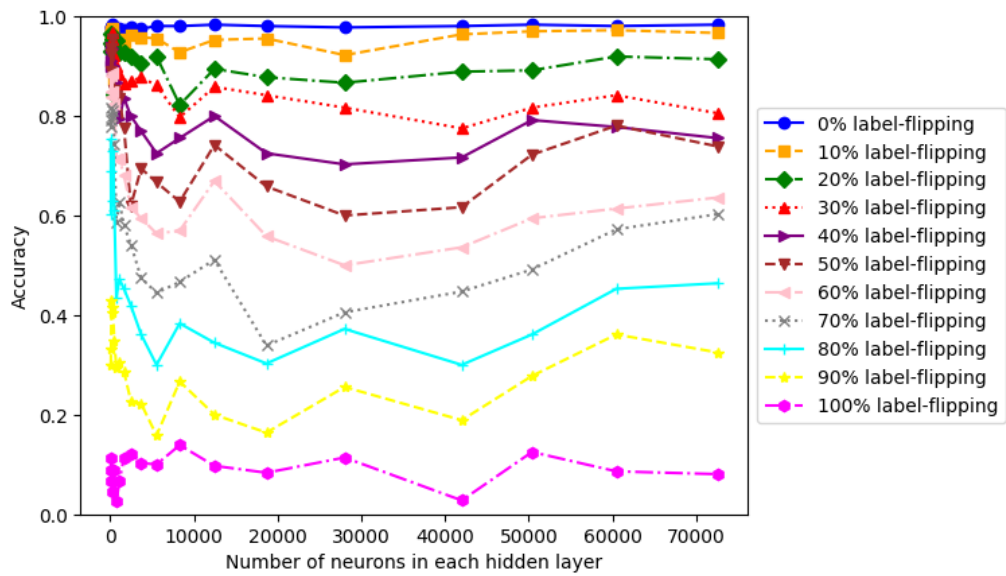


FIGURE 7 – *Accuracy* d'un modèle de réseau neurone profond en cas d'attaques par *label-flipping* avec différents nombres de neurones dans chaque couche cachée.

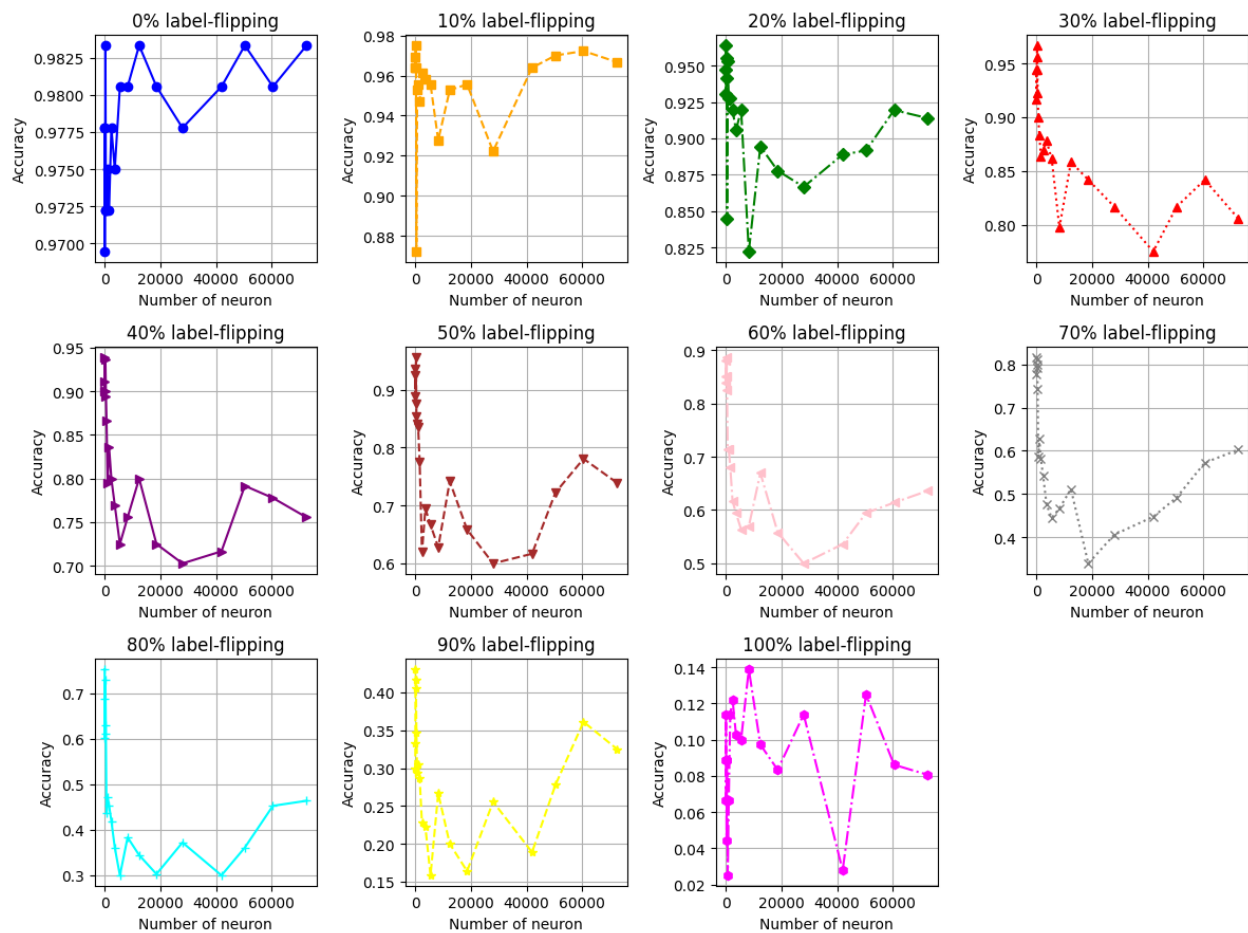


FIGURE 8 – Même résultat que la première image, nous avons simplement séparé les courbes pour mieux les visualiser

Analyse des résultats

Nous constatons tout d'abord qu'il existe également un décalage presque constant entre les différentes courbes pour divers taux de *label flipping*, comme c'est le cas pour les SVM et la régression. Ce décalage s'intensifie progressivement.

Nous observons ensuite que lorsque le nombre de neurones augmente, la précision du modèle fluctue, alternant entre améliorations et dégradations, à la manière d'une oscillation. Effectivement, la performance d'un réseau de neurones dépend de l'architecture des couches et du nombre de neurones dans chaque couche. Ainsi, augmenter le nombre de neurones peut parfois renforcer l'architecture du modèle, permettant une meilleure prédiction sur l'ensemble de test. Cependant, cela peut aussi complexifier l'architecture du modèle et conduire à un sur-apprentissage (*over-*

fitting) sur l'ensemble d'entraînement, réduisant potentiellement la précision.

En examinant les courbes individuellement, nous remarquons que pour chaque taux de *label flipping* allant de 0,3 à 0,9, la courbe de précision chute rapidement avant de ralentir aux alentours de 9 000 neurones. La précision remonte par la suite, puis redescend à nouveau au niveau de 20 000 neurones, avant d'augmenter à nouveau après 40 000 neurones. Cela indique qu'en ce qui concerne un réseau de neurones, accroître la taille du modèle n'entraîne pas systématiquement un effet négatif sur la précision.

Par conséquent, avec le modèle de réseau de neurones, les résultats sont contraires à notre conjecture initiale (quand de dimension augmente, *accuracy* augmente et diminue à partir d'un point maximum).

4.4 Premières conclusions

Nos observations remettent en question les conjectures initiales, notamment en ce qui concerne l'effet de l'augmentation dimensionnelle sur la performance des modèles. Les résultats montrent des comportements variés selon le type de modèle étudié :

- **La performance du modèle en fonction de la dimension :**
 - **Modèles de régression et de réseau de neurones :** Dans les modèles de régression et de réseau de neurones, les observations sont contraires à notre conjecture initiale. Nous observons que, pour le cas de la régression, *accuracy* du modèle augmente avec la dimension jusqu'à atteindre un pic, puis elle diminue. Cependant, ce pic de *accuracy* n'est pas toujours suivi d'une baisse significative car il augmente à nouveau à partir de certain point quand il y a de augmentation des dimensions. De même, pour les réseaux de neurones, *accuracy* n'est pas stable, parfois il augmente et parfois il diminue quand on augmente la dimension de modèle.
 - **Modèles SVMs :** Les résultats sont plus alignés avec notre hypothèse initiale pour mini-MNIST, où *accuracy* augmente puis diminue avec l'augmentation de la dimension, suivant approximativement une relation inverse à la racine carrée de la dimension. Cependant, les résultats de la seconde série de tests montrent une diminution linéaire, qui ne correspond pas à notre hypothèse initiale.
- **Effet des empoisonnements avec la performance du modèle :** Avec l'effet du empoisonnement sur le modèle, on remarque qu'il respecte notre conjecture. On voit que lorsque le taux de l'empoisonnement augmente, *accuracy* du modèle diminue dans les trois cas. De plus, on observe qu'il y a une translation de la courbe de *accuracy* d'un taux de l'empoisonnement à l'autre. Pour les modèles de régression et les réseaux de neurones, on constate que le taux de translation augmente de plus en plus à mesure que la précision augmente.

5 Étude mathématique des attaques

Nous allons désormais étudier mathématiquement le *label flipping*.

On souhaite étudier l'influence de données biaisées sur la performance d'un modèle confronté à des données réelles. On considérera donc que l'on attaque uniquement le *training set*, et on testera l'*accuracy* du modèle sur le *testing set*.

Pour cela, simplifions le modèle.

On considère que les *labels* sont dans $\mathbb{Y} = \{0, 1\}$.

Les données sont des éléments de $\mathbb{R}^d \times \mathbb{Y}$.

Il y en a N qu'on les partage en deux sous-ensembles, contenant respectivement de H et P données :

- $\mathbb{H} = (X_n, y_n) \in (\mathbb{R}^d \times \mathbb{Y})^H$ qui contient des données "saines" (*honest*). Leurs étiquettes sont correctes.
- $\mathbb{P} = (X_{H+k}, \tilde{y}_{H+k}) \in (\mathbb{R}^d \times \mathbb{Y})^P$ avec $\tilde{y}_{H+k} = 1 - y_{H+k}$ (y_{H+k} est le label réel) qui contient les données "empoisonnées" (*poisoned*). Leurs étiquettes ont été remplacées.

On effectue une régression logistique, avec comme perte la *cross-entropy loss*. Le gradient de la *perte* s'écrit, en notant α le vecteur de poids et σ la fonction sigmoïde :

$$\begin{aligned} \vec{\nabla} L &= \sum_{\mathbb{H}} (\sigma(\alpha^T x_n) - y_n) x_n + \sum_{\mathbb{P}} (\sigma(\alpha^T x_{H+k}) - \tilde{y}_{H+k}) x_{H+k} \\ &= \sum_{\mathbb{N}} (\sigma(\alpha^T x_n) - y_n) x_n - \sum_{\mathbb{P}} (\sigma(\alpha^T x_{H+k}) - y_{H+k}) x_{H+k} + \sum_{\mathbb{P}} (\sigma(\alpha^T x_{H+k}) - \tilde{y}_{H+k}) x_{H+k} \\ &= \sum_{\mathbb{N}} (\sigma(\alpha^T x_n) - y_n) x_n + \sum_{\mathbb{P}} (2y_{H+k} - 1) x_{H+k} \end{aligned}$$

Il apparaît que le *label flipping* se rapproche d'une attaque de gradient dans son effet : on ajoute au gradient honnête un terme perturbateur (noté ici C) qui peut, si bien choisi, faire pointer le gradient total vers une direction dévastatrice (une direction qui va augmenter la *loss* au lieu de la diminuer). On a :

$$C = \sum_{\mathbb{P}} (\sigma(\alpha^T x_{H+k}) - \tilde{y}_{H+k}) x_{H+k}$$

Cela fait apparaître la contrainte $C \in \text{Vect}((x_{H+i})_{i \in [1, P]})$.

Le *label flipping* est donc moins flexible que l'attaque de gradient, qui permet de contrôler directement et à chaque étape la perturbation que l'on inflige au gradient "honnête". Cela permet un bien meilleur potentiel d'attaque que le *label flipping*, certes très facile à implémenter, mais qui ne permet pas d'attaquer aussi directement et efficacement le gradient. En effet, effectuer un *label flipping* se fait en principe en amont, et une fois $\mathbb{P}$ défini, le champ de gradient est modifié de façon fixe.

Pour pouvoir faire l'attaque la plus dévastatrice possible, l'attaquant doit connaître tout sur le modèle à chaque instant.

Définition : Attaquant omniscient : Attaquant qui connaît les paramètres et le gradient à chaque pas.

Changeons le mode d'attaque : l'attaquant est désormais omniscient, et à chaque étape, définit P de façon à attaquer le plus efficacement possible le gradient. On cherche donc la donnée (x_P, y_P) dont la contribution au gradient est la plus agressive envers le gradient produit par H (il suffira alors de prendre P fois cette donnée).

Reprenons la formule précédemment obtenue, en ramenant $\mathbf{P}$ à ce seul point :

$$\tilde{\nabla} L = \sum_{\mathbf{H}} (\sigma(\alpha^T x_n) - y_n) x_n + (\sigma(\alpha^T x_P) - y_P) x_P$$

La perturbation s'écrit donc $(\sigma(\alpha^T x_P) - y_P) x_P$.

Maximiser l'effet de la perturbation sur le gradient "honnête" revient à minimiser le produit scalaire :

$$\langle \sum_{\mathbf{H}} (\sigma(\alpha^T x_n) - y_n) x_n, (\sigma(\alpha^T x_P) - y_P) x_P \rangle$$

À une certaine étape de la descente de gradient, on souhaite sélectionner (x_P, y_P) dans l'espace des possibilités de façon à minimiser ce produit scalaire en fonction de α (le vecteur des poids) à cette étape.

$$\begin{aligned} \sigma(\alpha^T x_P) - y_P &\in (-1, 0) & \text{si } y_P = 1 \\ \sigma(\alpha^T x_P) - y_P &\in (0, 1) & \text{si } y_P = 0 \end{aligned}$$

La perturbation sera intuitivement la meilleure si elle est de même direction que le gradient et de sens opposé, ou orthogonale à ce gradient. On peut imaginer donc deux méthodes :

5.1 Méthode d'attaque colinéaire :

On souhaite donc prendre x_P colinéaire à

$$\nabla \tilde{L}_H = \sum_{\mathbf{H}} (\sigma(\alpha^T x_n) - y_n) x_n$$

et de même sens si $y_P = 1$, de sens opposé si $y_P = 0$.

Cette direction n'est en pratique pas toujours accessible. On présuppose en effet que les caractéristiques du *dataset* permettent de choisir un tel point dans l'espace, ce qui n'est pas toujours faisable (par exemple, si les données sont dans l'hypercube $[0, 1]^3$, on ne pourra prendre $x_P = (2, -5, 7)$). Dans un tel cas, il faut chercher un x_P compatible avec le *dataset* qui soit le plus proche possible de cette direction, voire étudier la possibilité de sélectionner différents points empoisonnés dont la somme des perturbations respectives se rapproche de la direction recherchée. Cela doit se faire au cas par cas selon le *dataset* et dépasse le cadre de cette étude.

La première stratégie d'attaque sera donc d'attaquer le gradient colinéairement. Dans le cas où cette direction n'est pas accessible, on peut essayer de l'attaquer orthogonalement (cela sera plus pratique à implémenter).

5.2 Méthode d'attaque orthogonale :

On prend x_p tel que $\langle \vec{\nabla} L_H, x_p \rangle = 0$, c'est à dire on prend x_p orthogonal à $\vec{\nabla} L_H$.

On note $l = \lim_{\lambda \rightarrow +\infty} \sigma(\alpha^T \cdot (\lambda \cdot x_p))$

On prend $y_p = 1$ si $l < 0.5$ et $y_p = 0$ sinon, afin de maximiser $|l - y_p|$.

donc

$$(\sigma(\alpha^T \cdot (\lambda \cdot x_p)) - l) \cdot \lambda \cdot x_p \sim_{\lambda \rightarrow +\infty} (l - y_p) \cdot \lambda \cdot x_p$$

or $(l - y_p) \neq 0$

Donc

$$\vec{\nabla} L(\lambda) \sim_{\lambda \rightarrow +\infty} (1 + \lambda \cdot (l - y_p)) \cdot x_p$$

Donc on prenant λ très grand, on peut faire diverger $\vec{\nabla} L$ vers un vecteur orthogonal à $\vec{\nabla} L_H$ de norme assez grande.

Il reste donc la contrainte de normalisation, auquel cas il faut prendre le λ qui maximise $\|\lambda \cdot x_p\|_1$ tel que $\|\lambda \cdot x_p\|_1 \leq 1$, c'est-à-dire prendre $\lambda = \pm \frac{1}{\|x_p\|_1}$

Les deux stratégies d'attaque seront testées expérimentalement dans la partie suivante, afin de comparer leurs efficacités.

6 Expériences sur les attaques

Dans les séries d'expérience suivante, nous testons un modèle de régression logistique avec la Cross Entropy Loss, sur le dataset MNIST, mais seulement pour classer si les images représentent des 1 ou non. On suppose dans ces expériences que l'attaquant peut changer à chaque itération les données empoisonnées et a accès aux paramètres du modèle en lecture uniquement. Voici les détails des expériences :

6.1 Description de l'expérience

Langage utilisé : Python

Librairie utilisée : numpy, pytorch, datasets

Sauvegarde des données : Manuellement avec des listes python

Paramètres : On dispose du type d'attaque (Label Flipping normale, méthode colinéaire ou méthode orthogonale), et du poison rate comme paramètres variables.

Modèle : Régression linéaire suivie par une fonction sigmoïde σ

Algorithme : Voici l'algorithme utilisé pour l'expérience

1. **Adapter le dataset :** Comme vu précédemment, la première partie de l'algorithme consiste à transformer l'ensemble des images en vecteurs normalisés. On commence par aplatir les images, c'est-à-dire les transformer d'une matrice de luminosité en vecteurs, en concaténant les lignes de la matrice. Ensuite on normalise en divisant par 256 (Les valeurs initiales étaient de 1 à 256).
2. **Label Flipping :** Cette partie est ignorée pour les autres types d'attaque. Elle effectue la même transformation vu précédemment :

$$\forall i \in \llbracket 1, [r.N] \rrbracket : y_i \leftarrow (y_i + 1) \mod 10$$

3. **Entraînement :** L'entraînement consiste à exécuter un algorithme d'optimisation sur plusieurs itérations, l'algorithme utilisé ici est la *Stochastic Gradient Descent*, aussi notée SGD avec un learning rate de 0.01. Le nombre d'itérations est fixé à 1500. chaque itération, on calcule les points empoisonnés (même point calculé répété $[r.N]$ fois) selon la méthode choisie. Pour la méthode orthogonale, étant donné l'hyperplan d'où on peut prendre le vecteur x_p , pour simplifier les calculs, on choisit le vecteur de la manière suivante. Soit $u = \vec{\nabla}.L_H$. On choisit $k = \operatorname{argmax}_i |u_i|$. $\forall i \neq k : v_i = u_i$ et :

$$v_k = -\frac{\sum_{i \neq k} u_i^2}{u_k}$$

On a pris $k = \operatorname{argmax}_i |u_i|$ pour avoir, après normalisation, v à valeurs non négligeables.

Le vecteur de l'attaque est ensuite normalisé pour avoir des valeurs dans $[-1, 1]$

4. **Évaluation des performances :** Pour évaluer les performances, après chaque 50 itérations, on sauvegarde la test accuracy. Pour évaluer l'accuracy, on prend le label sortant ayant une valeur dans $[0, 1]$, et on prend $\operatorname{argmin}_{i \in \{0,1\}} |y - i|$, qu'on vérifie alors avec le vrai label.

6.2 Exécution et résultats des expériences

Nous avons exécuté l'algorithme précédent pour du Label Flipping normal et les deux méthodes, pour des poison rate $r \in \{0, 0.05, 0.1, \dots, 0.45\}$. La courbe suivante affiche les test accuracies de chaque test.

Voici les courbes résultantes :

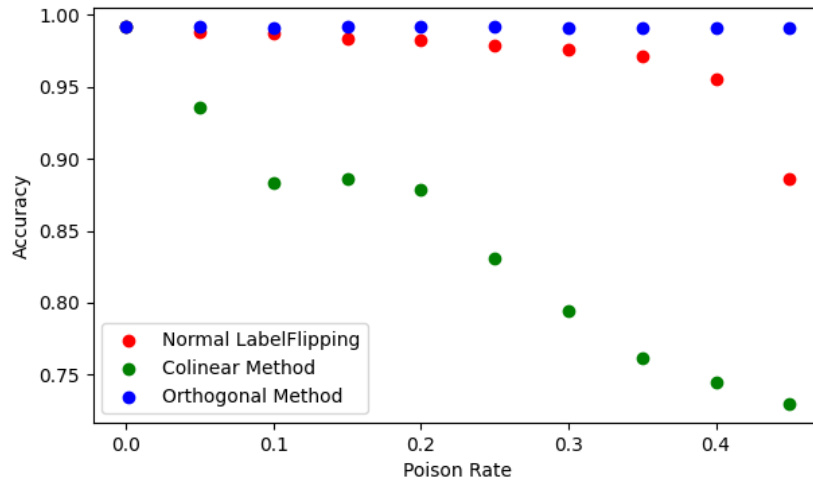


FIGURE 9 – Évolution de la test accuracy en fonction du *poison rate* et de la méthode d'attaque : *label flipping* normal, Méthode colinéaire, Méthode orthogonale.

Interprétation

On remarque d'après la courbe que la méthode orthogonale n'a pas d'effet, tandis que le méthode colinéaire est plus efficace que du *Label Flipping* normal.

Voici une autre courbe analysant plus en détails les effets de la méthode orthogonale en comparaison avec un cas sans attaque. Ces deux courbes montre l'évolution de la test accuracy en fonction du nombre d'itérations d'entraînements.

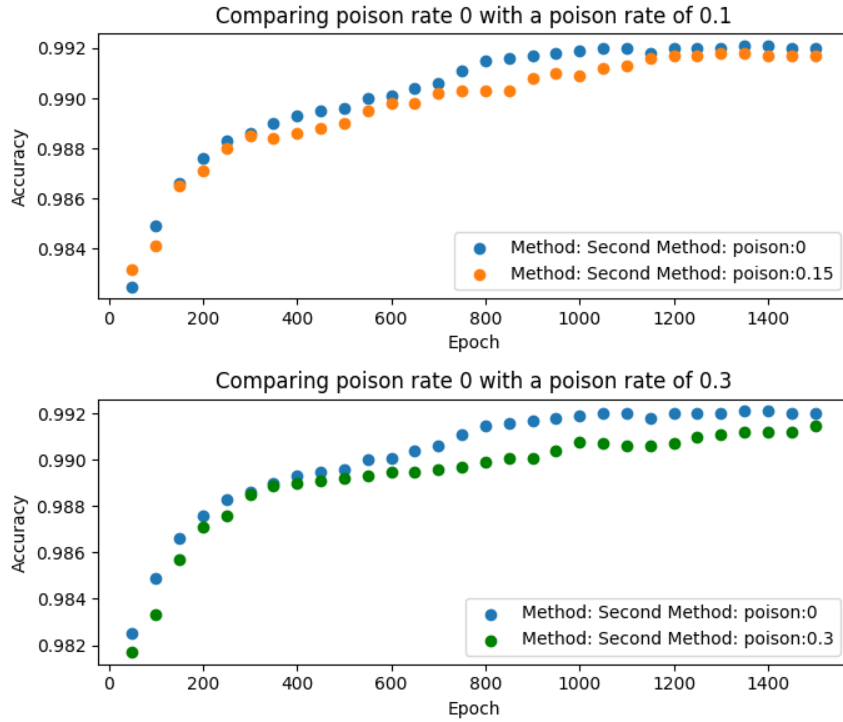


FIGURE 10 – Évolution de la test accuracy en fonction du nombre d’itérations pour différentes valeurs du *poison rate*.

Analyse et interprétation

On peut remarquer un léger ralentissement lors de l’entraînement. Donc on peut dire que la méthode orthogonale a un petit effet sur la vitesse de convergence mais par forcément sur le point de convergence.

Cependant, on ne peut pas conclure si la méthode orthogonale est véritablement inefficace. En effet, la dimension de choix est $m - 1$ où m est le nombre de paramètre du modèle. Dans notre expérience, on a juste changer un des paramètre, donc il est probable que cela soit la raison de l’inefficacité aperçue. Cela laisse place à une nouvelle recherche sur le vecteur orthogonal optimale pour attaquer le modèle.

6.3 Synthèse des observations

Les expériences antérieures démontrent une nette supériorité de l’efficacité de la méthode colinéaire par rapport à celle du *Label Flipping* normal. Cette efficacité est obtenue même avec une normalisation des vecteurs, ce qui la rend praticable dans des cas réels, puisque les données sont en général normalisées. Cependant, une difficulté subsiste : l’hypothèse selon laquelle un attaquant peut modifier les valeurs des ensembles de données à chaque itération est difficilement réalisable.

Il est également à noter que la méthode orthogonale nécessite des recherches approfondies, en particulier pour déterminer la manière optimale de choisir le vecteur orthogonal.

7 Faiblesse des modèles sans défense

La vulnérabilité du modèle face à diverses attaques est en grande partie due aux méthodes d'agrégations, l'agrégation étant la manière dont on rassemble les gradients des loss en chaque point ou ensemble de points. Par exemple, dans l'analyse précédente, on a utilisé comme fonction d'agrégation la moyenne, ce qui nous a permis d'attaquer avec facilité (dans le cadre d'un attaquant omniscient) le modèle.

Pour optimiser les modèles et les entraîner, la plupart utilise une descente de gradient (par exemple SGD : stochastic gradient descent) qui visent à minimiser une fonction de coût (loss function $\mathcal{L}$). En général, cette fonction de coût totale est obtenue en prenant la moyenne des fonctions de coût sur l'ensemble du jeu de données. Bien que cette moyenne soit généralement précise, elle peut être facilement perturbée. En revanche, la médiane offre une mesure plus robuste.

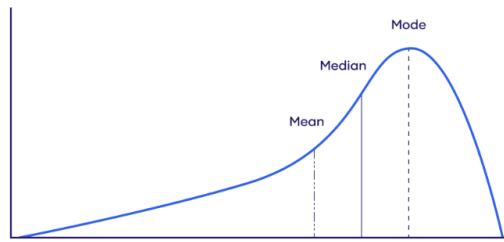


FIGURE 11 – Moyenne vs Médiane

Dans la figure suivante, on a un point qui est assez loin du reste du *dataset*. On remarque que la moyenne est plus décalée, mais que la médiane reste inchangée.

Cette faiblesse de la médiane rend l'entraînement des modèles assez faible en terme de robustesse. En effet, une entrée du *dataset* qui est assez éloignée du reste peut considérablement affecter la performance.

Notamment, pour l'attaque de gradient, un seul gradient suffirait à détruire le modèle.

Les défenses contre les attaques d'empoisonnement optimal consistent généralement soit à identifier les exemples malveillants (par exemple une détection de points aberrants) et à les écarter des données d'entraînement [8], soit à résoudre un problème d'optimisation robuste [3, 4].

Une méthode employée pour améliorer la robustesse d'un modèle face à de telles perturbations est la médiane géométrique, qui est définie comme ceci (en ra-

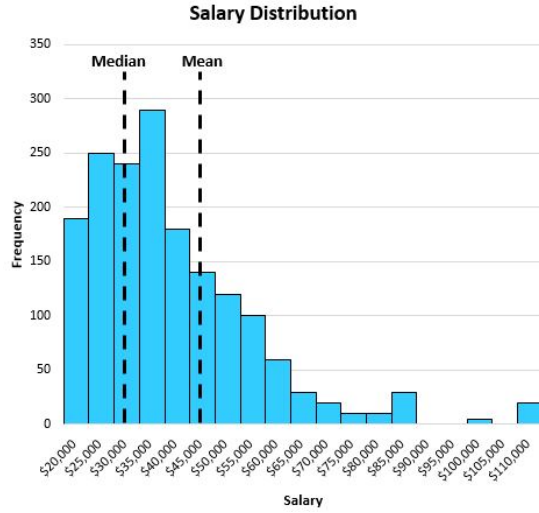


FIGURE 12 – Moyenne vs Médiane : présence de valeurs aberrantes

joutant le point poison (x_p, y_p) :

$$\text{GEOMED}((u_i)_{i \in [1, \dots, n]}) := \arg \min_v \sum_{i=1}^n \|v - u_i\| = \arg \min_v \left(\sum_H \|(\sigma(\alpha^T x_n) - y_n)x_n - v\| + \|(\sigma(\alpha^T x_p) - y_p)x_p - v\| \right)$$

Notons la GEOMED Honnête :

$$\text{GEOMED}^{(H)} = \arg \min_v \left(\sum_H \|(\sigma(\alpha^T x_n) - y_n)x_n - v\| \right)$$

Normalement, on doit trouver poison (x_p, y_p) qui va faire "éloigner" le plus possible GEOMED de $\text{GEOMED}^{(H)}$

On aimerait également étendre l'étude à la médiane géométrique, pour le moment assez délicat. Seule idée : empoisonner de la même manière que pour la moyenne. Reste à étudier expérimentalement que l'effet est bien optimal ou proche de l'optimal.

En bref il faut faire le pont avec la partie expérimentale.

L'attaquant est omniscient et capable d'interférer à chaque pas de la descente du gradient.

Proposition (Lopuhaa, 1989; El-Mhamdi et al., 2023; Lê Nguyễn Hoàng, 2024) [6]
Soient $H > P$. Soit $g^{(H)} = \text{GEOMED}(x_1, \dots, x_H)$ la médiane géométrique des vecteurs honnêtes Alors pour tout ensemble de vecteurs poisons $(x_{H+1}, \dots, x_{H+P})$ et toute médiane géométrique empoisonnée $g = \text{GEOMED}(x_1, \dots, x_H, x_{H+1}, \dots, x_{H+P})$,

$$\|g - g^{(H)}\|_2 \leq \left(1 - \frac{P^2}{H^2}\right)^{-\frac{1}{2}} \max_{h \in [H]} \|x_h - g^{(H)}\|_2.$$

Donc

$$\|g^n - g^{(n-1)}\|_2 \leq \left(1 - \frac{1}{H^2}\right)^{-\frac{1}{2}} \max_{h \in [H] \cup \{x_{(H+1)}, \dots, x_{(H+n-1)}\}} \|x_h - g^{(H)}\|_2.$$

D'où

$$\|g^{(n)} - g^{(H)}\|_2 \leq \left(\sum_{i=0}^{n-1} \left(1 - \frac{1}{(H+i)^2}\right)^{-\frac{1}{2}}\right) M$$

Où $M = \max_{h \in [H] \cup \{x_i\}_{i \in P}} \|x_h - g^{(H)}\|_2$.

Donc à chaque pas on rajoute un point (donc $P=1$ à cet instant), les points honnêtes étant :les H réels + les points poisons qu'on a rajouté jusqu'à présent (qui ont été incorporés au *dataset* donc forment un nouveau H) . Notons g^n la médiane géométrique au pas n

En supposant $H \geq 2$, ce qui est une hypothèse raisonnable, on a, pour tout $i \in \mathbb{N}$:

$$1 + 1/2 \frac{1}{(H+i)^2} \leq \left(1 - \frac{1}{(H+i)^2}\right)^{-\frac{1}{2}} \leq 1 + \frac{1}{(H+i)^2}$$

De cet encadrement, on en déduit l'équivalent :

$$\sum_{i=0}^{n-1} \left(1 - \frac{1}{(H+i)^2}\right)^{-\frac{1}{2}} \sim n$$

Donc ce type d'attaques aura l'effet d'une perturbation au plus linéaire en fonction du nombre d'itérations.

8 Conclusion

En résumé, cette analyse approfondie de la robustesse des modèles de machine learning a fourni des éclairages précieux sur plusieurs aspects cruciaux. Tout d'abord, l'impact de la dimension sur les performances des modèles lors d'attaques telles que le *label flipping* a été minutieusement examiné. Les résultats ont démontré de manière convaincante que la dimensionnalité n'exerce qu'une influence marginale sur la robustesse des modèles, soulignant ainsi l'importance de considérer d'autres facteurs dans l'évaluation de la sécurité des modèles.

De même, nos efforts pour élaborer des attaques de *label flipping* en nous inspirant de l'attaque par gradient ont porté leurs fruits, avec des résultats prometteurs obtenus pour l'une des méthodes explorées. Cette avancée suggère des pistes intéressantes pour améliorer notre compréhension des vulnérabilités potentielles des modèles de machine learning et pour développer des contre-mesures appropriées.

Enfin, notre analyse théorique de la défense des modèles à l'aide de la médiane géométrique a ouvert de nouvelles perspectives dans la conception de mécanismes de protection contre les attaques. En intégrant cette approche dans les stratégies de défense, il est possible d'améliorer considérablement la robustesse des modèles face à diverses menaces.

En conclusion, cette étude souligne l'importance cruciale de la recherche continue dans le domaine de la sécurité des modèles de machine learning. En identifiant les failles potentielles et en développant des solutions innovantes, nous pouvons progresser vers des systèmes plus fiables et sécurisés, capables de répondre aux défis croissants de la cybersécurité dans le domaine du machine learning.

Références

- [1] P. Blanchard, E. El-Mhamdi, R. Guerraoui, and J. Stainer. Machine learning with adversaries : Byzantine tolerant gradient descent. In *Neural Information Processing Systems*, pages 118–128, 2017.
- [2] E.-M. El-Mhamdi, S. Farhadkhani, R. Guerraoui, N. Gupta, L.-N. Hoang, R. Pinot, and J. Stephan. On the impossible safety of large ai models. *arXiv preprint arXiv :2209.15259*, 2022.
- [3] J. Feng, H. Xu, S. Mannor, and S. Yan. Robust logistic regression and classification. In *Neural Information Processing Systems*, 2014.
- [4] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li. Manipulating machine learning : Poisoning attacks and countermeasures for regression learning. 2021.
- [5] A. Kattis and A. Nikolov. Lower bounds for differential privacy from gaussian width. In B. Aronov and M. J. Katz, editors, *33rd International Symposium on Computational Geometry, SoCG 2017, July 4-7, 2017, Brisbane, Australia*, volume 77 of *LIPIcs*, pages 45 :1–45 :16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017.
- [6] Lê-Nguyên. The poison of dimensionality. *Article en cours de révision à la Neural Information Processing Systems - Communication privée de l'article via notre tuteur El Mahdi El Mhamdi*, 2024.
- [7] E. M. E. Mhamdi, S. Farhadkhani, R. Guerraoui, A. Guirguis, L.-N. Hoang, and S. Rouault. Collaborative learning in the jungle (decentralized, byzantine, heterogeneous, asynchronous and nonconvex learning). In *Thirty-Fifth Conference on Neural Information Processing Systems*, 2021.
- [8] A. Paudice, L. Muñoz-González, A. Gyorgy, and E. C. Lupu. Detection of adversarial training examples in poisoning attacks through anomaly detection. 2018.

Annexe

Code des expériences

Voici le lien vers le dépôt GitHub contenant les codes des expériences. Beetorch
Repository Link